

上海大学 计算机工程与科学学院

《数据库原理》(1) 实验报告

姓名 赵文宇 学号 23121769

时间 一 3-4 地点 计 706 指导教师 李晓强

实验 3: SQL 命令-单表查询和多表查询

一、实验目的

- 1、了解和掌握 SQL 命令的单表查询方法（包括逻辑表达式）
- 2、了解和掌握 SQL 命令的多表查询方法

二、实验内容

- 1、lect03 讲稿中的多表查询
- 2、对 school 数据库做如下查询：
 - (1) 验证在 1000 万个以上记录时在索引和不索引时的查询时间区别。
 - (2) 查询 2011 年进校年龄大于 20 岁的男学生的学号与姓名。
 - (3) 检索刘晓明不学的课程的课程号。
 - (4) 检索马小红老师所授课程的学年，学期，课程号，上课时间。
 - (5) 查询计算机学院男生总评成绩及格、教授开设的课程的课程号、课名、开课教师姓名，按开课教师升序，课程号降序排序。
 - (6) 检索学号比张颖同学大，年龄比张颖同学小的同学学号、姓名。
 - (7) 检索同时选修了“08305001”和“08305002”的学生学号和姓名。

三、实验过程

（描述实践的过程和结果、遇到的问题和解决方案）

首先是多表查询，因为我们小组上周的研讨主题正好是用四种方式查询马小红老师教的课，可以说是涵盖了基本所有的重要多表查询方式，因此在这里仅对部分句式进行解释，且以后面的任务(4)进行例子写 sql 语句，因为给出的四种方式也差不多涵盖了所有的多表查询技巧。

第一种就是传统的引入多个表格，用间接链接的方法进行查询的。

```
SELECT O.xq, O.kh, O.sksj FROM T, O
WHERE T.gh = O.gh AND T.xm = '马小红';
```

第二种是嵌套方式查询，sql 语句如下所示：

```
SELECT xq, kh, sksj FROM O
WHERE gh IN (SELECT gh FROM T WHERE xm = '马小红');
```

第三种是用关系词 exists 查询，如下所示：

```
SELECT xq, kh, sksj FROM O
WHERE EXISTS (SELECT * FROM T WHERE T.gh = O.gh AND T.xm =
'马小红');
```

最后一种则是将两张比较进行“合并”，即 JOIN 链接。

```
SELECT O.xq, O.kh, O.sksj FROM O
JOIN T ON O.gh = T.gh
WHERE T.xm = '马小红';
```

运行结果皆为：

	xq	kh	sksj
1	2012-2013 冬季	08305002	星期三 1-4
2	2012-2013 冬季	08305003	星期五 5-6
3	2013-2014 秋季	08305001	星期一 5-6

(1) 验证在 1000 万个以上记录时在索引和不索引时的查询时间区别：

我选用的是 S 表作为基础表格进行扩容、查询。为了不影响后续 S 表的正常使用，我将 S 表复制了一份，作为新的 S_new 表，用于实验。经过在网上查了一些文章，发现可以通过复制追加的办法，用指

数级的次数即可生成一个记录数超过 1000w 次的表。由于初始记录数为 7，经计算可得 7×2^{21} 次就已经 >1000w 了，所以循环只需要 21 次即可。这边有一个 while 语句在外层即可。具体的 sql 语句如下图所示：

```
SELECT * INTO S_new FROM S;
DECLARE @i INT = 0;
WHILE @i < 21
BEGIN
    INSERT INTO S_new SELECT * FROM S_new;
    SET @i = @i + 1;
END
```

执行后，观察结果，已成功创建。

```
(14336 行受影响)
(28672 行受影响)
(57344 行受影响)
(114688 行受影响)
(229376 行受影响)
(458752 行受影响)
(917504 行受影响)
(1835008 行受影响)
(3670016 行受影响)
(7340032 行受影响)
```

先来测试没有索引的时候，sql 语句如下：

```
USE school
SET STATISTICS TIME ON;
SET STATISTICS IO ON;
SELECT * FROM S_new
WHERE xm = '张颖';
```

没有索引的时候，查询的时间为 5781ms

```
(2097152 行受影响)
表"S_new"。扫描计数 1，逻辑读取次数 104856，物理读取次数 0，页面服务器读取次数 0，预读读取次数 0，页面服务器预读读取次数 0

SQL Server 执行时间:
CPU 时间 = 5781 毫秒，占用时间 = 8092 毫秒。

完成时间: 2026-03-20T19:25:43.8451992+08:00
```

加入索引查询的语句如下所示：

```
CREATE INDEX idx_test_xm ON S_new(xm);
```

而加入了索引之后，查询的时间为 5250ms

(2097152 行受影响)
表“S_new”。扫描计数 1, 逻辑读取次数 104856, 物理读取次数 0, 页面服务器读取次数 0, 预读读取次数 0, 页面服务器预读读取次数 0
SQL Server 执行时间:
CPU 时间 = 5250 毫秒, 占用时间 = 8268 毫秒。
完成时间: 2026-03-20T19:33:11.7355764+08:00

虽然看上去没有很大的提升，但是经过计算也提升了 9.18%，在效率上还是有一定的优化的。

（由于篇幅问题后面几个小任务就不具体讲为什么这么做以及语句的逻辑了）

(2) 查询 2011 年进校年龄大于 20 岁的男学生的学号与姓名。

Sql 语句以及查询结果如下图所示：

The screenshot shows a SQL query in a text editor and its results in a table. The query is:

```
1 use school
2 select xh, xm
3 from S
4 where csrq < '1991-09-01' and xb='男'
```

Below the query, there is a status bar indicating 145% zoom and a green checkmark with the text "未找到相关问题". Below that, there is a tabbed interface with "结果" (Results) selected. The results are displayed in a table with two columns: "xh" (Student ID) and "xm" (Student Name).

	xh	xm
1	1105	刘成刚

(3) 检索刘晓明不学的课程的课程号。

Sql 语句以及查询结果如下图所示：

```

1      use school
2      select distinct kh
3      from C
4      where kh not in(
5          select distinct E.kh
6          from S,E
7          where S.xm='刘晓明' and S.xh=E.xh
8      )

```

145 % 未找到相关问题

结果 消息

	kh
1	08301001
2	08302001
3	08305003

(4) 检索马小红老师所授课程的学年，学期，课程号，上课时间。

Sql 语句以及查询结果如下图所示：

```

1      use school
2      select O.xq, O.kh, O.sksj
3      from O
4      JOIN T on O.gh = T.gh
5      where T.xm='马小红'

```

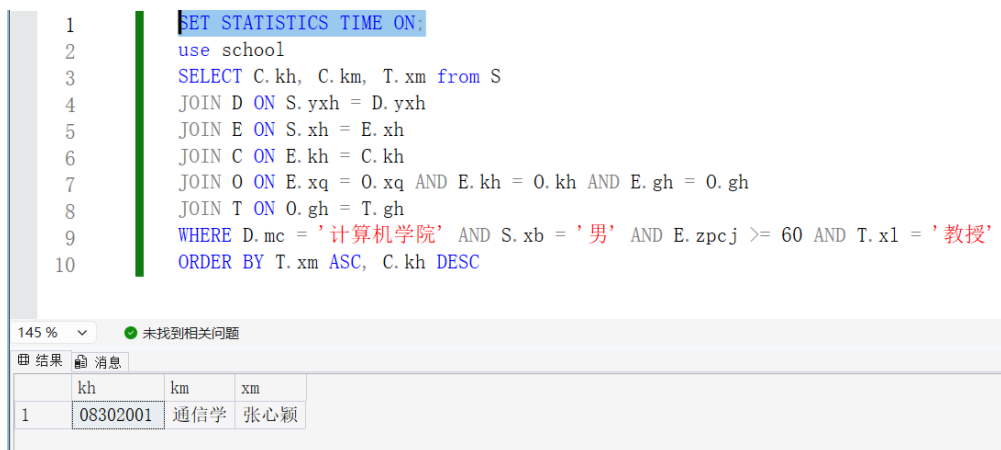
145 % 未找到相关问题

结果 消息

	xq	kh	sksj
1	2012-2013 冬季	08305002	星期三 1-4
2	2012-2013 冬季	08305003	星期五 5-6
3	2013-2014 秋季	08305001	星期一 5-6

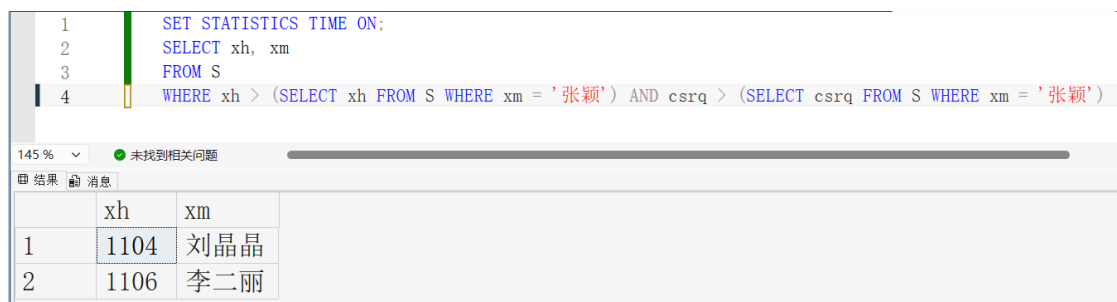
(5) 查询计算机学院男生总评成绩及格、教授开设的课程的课程号、课名、开课教师姓名，按开课教师升序，课程号降序排序。

Sql 语句以及查询结果如下图所示：



(6) 检索学号比张颖同学大，年龄比张颖同学小的同学学号、姓名。

Sql 语句以及查询结果如下图所示：



(7) 检索同时选修了“08305001”和“08305002”的学生学号和姓名。

Sql 语句以及查询结果如下图所示：



四、收获与感想

如果说实验二是对 SQL 语法的“初次邂逅”，那么实验三则是进入了数据库灵魂的“深层对话”。本次实验的跨度很大，从复杂的多表关联查询到千万级数据的性能调优，让我对数据库系统有了更宏观的认识。最令我兴奋的环节莫过于对 1000 万条记录进行索引验证。

起初我对“索引”的概念仅停留在理论书本，但当我亲手编写 WHILE 循环将数据量呈指数级翻倍，并亲眼看到加上索引后查询时间缩短了 9.18% 时，感觉到还挺不可思议的，因为这个指标被量化了。此外，在处理马小红老师授课任务的四种查询方式时，我发现无论是传统的连接查询、嵌套子查询，还是 EXISTS 关系词，都能达到“殊途同归”的效果。这让我明白 SQL 并不是死板的公式，而是一门充满灵活性的语言。在处理如“计算机学院及格男生”这类多条件过滤时，复杂的逻辑判断考验了我的细心程度，也让我养成了在编写复杂 SQL 前先理清表间关联关系的习惯。这次实验不仅让我掌握了多表连接、嵌套等进阶技巧，还让我对于查找有了更进一步的认识，尤其是在多表查询的要求中。